

---

**Bases teóricas de estructuras de almacenamiento en memoria**  
**GA3-220501093-AA3-EV01**



**Aprendiz:**  
**Johan Mauricio Sánchez Gaviria**

**Instructor:**  
**Carlos Alberto Fuel**

**Servicio nacional de aprendizaje el Sena**

**Análisis y desarrollo del software**  
**Ficha 3134555**

## **Introducción**

A continuación, exploraremos algunos conceptos fundamentales de JavaScript, incluyendo diferencias, tipos de datos, operadores, entre otros aspectos. Este conocimiento nos ayudará a comprender mejor JavaScript, un lenguaje crucial entre otros, y nos proporcionará una visión clara de su composición. Además, respetaremos otros conceptos útiles que son relevantes en el proceso de programación.

## Diferencias entre los lenguajes compilados e interpretados

| COMPILADOS                                                  | INTERPRETADOS                                                   |
|-------------------------------------------------------------|-----------------------------------------------------------------|
| Necesita que antes de ser ejecutado pueda ser código fuente | Traduce las instrucciones cada vez que es ejecutado             |
| Son optimizados en el momento de ser ejecutado              | Debe tener instalado el interpretador                           |
| El compilado se puede ejecutar por sí solo                  | Siempre necesita su intérprete para ser ejecutado               |
| Genera un archivo binario no modificable                    | Definido y modificable en cada momento                          |
| Las instrucciones vienen enviadas directamente al procesado | Las instrucciones deben traducirse antes de llegar al procesado |
| Se puede ejecutar en ciertas máquinas y sistemas operativos | Funciona en todas las máquinas y sistemas                       |
| Lenguajes compilados son C, C++, Delphi                     | lenguajes interpretados son Python, Javascript, Perl, Php       |

## Diferencias de JavaScript

- Es esencial para el desarrollo de aplicaciones web y también para las aplicaciones móviles.
- Es más sencillo tanto para la máquina como para el programador.
- Se trata de un lenguaje interpretado.
- Se integra directamente en HTML.
- Permite que el código funcione mientras se ejecuta y muestra los errores al mismo tiempo.
- Puede contener texto, números, arrays u objetos.

## Tipos de datos primitivos y uso de javascript

- **Undefined:** representa un valor indeterminado o indefinido
- **Boolean:** es un tipo de dato booleano donde los valores posibles son true o false.
- **Number:** representa números enteros o decimales.
- **String:** representa cadenas de texto.
- **BigInt:** representa números enteros grandes.
- **Symbol:** representa una referencia a otros datos.

**Tipos de declaración de variables:**

- **var:** declara una variable y opcionalmente la inicializa con un valor. Tiene un alcance de función.
- **let:** declara una variable local con ámbito de bloque y opcionalmente la inicializa con un valor.
- **const:** declara una constante de solo lectura con ámbito de bloque.

#### Otros tipos de datos:

- **Null:** representa la ausencia de valor y se muestra cuando el objeto no se hereda.
- **Object:** utilizado con la palabra clave new para crear instancias de objetos como Array, Map, Set, WeakMap, WeakSet, Date, entre otros
- **Function:** una estructura que no contiene datos, aunque también responde al operador typeof.

### Operadora de javascript

Los Operadores de Asignación permiten asignar un valor a la variable de la izquierda basándose en el valor de su operando derecho. El operador más simple es (=) que se utiliza para la asignación básica. A continuación, se describen otros operadores de asignación disponibles:

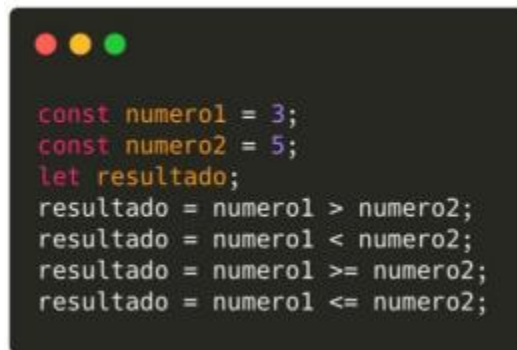
- Asignación de adición (+=): Añade el valor del operando derecho al valor de la variable.
- Asignación de resta (-=): Resta el valor del operando derecho al valor de la variable.
- Asignación de multiplicación (\*=): Multiplica el valor de la variable por el valor del operando derecho.
- Asignación de división (/=): Divide el valor de la variable por el valor del operando derecho.
- Asignación de residuo (%=): Asigna el residuo de la división del valor de la variable por el valor del operando derecho
- Asignación de exponenciación (\*\*=): Asigna el resultado de elevar el valor de la variable a la potencia del operando derecho.

```
const variableA = 5;
let variableB = 'Hola';
variableB = 10;
```

## Los Operadores de Comparación o Relacionales

se utilizan para comparar dos expresiones y el resultado puede ser falso o verdadero (booleano). A continuación, se describen los operadores de comparación disponibles:

- **Igual (==):** Comprueba si dos valores son iguales.
- **No es igual (!=):** Comprueba si dos valores no son iguales.
- **Estrictamente igual (===):** Comprueba si dos valores son iguales en valor y tipo de datos.}
- **Desigualdad estricta (!==):** Comprueba si dos valores no son iguales en valor o tipo de datos.
- **Mayor que (>):** Comprueba si el primer valor es mayor que el segundo.
- **Mayor o igual que (>=):** Comprueba si el primer valor es mayor o igual que el segundo.
- **Menor que (<):** Comprueba si el primer valor es menor que el segundo.
- **Menor o igual que (<=):** Comprueba si el primer valor es menor o igual que el segundo.



```
const numero1 = 3;
const numero2 = 5;
let resultado;
resultado = numero1 > numero2;
resultado = numero1 < numero2;
resultado = numero1 >= numero2;
resultado = numero1 <= numero2;
```

## Operadores aritméticos

Toma valores numéricos como sus operadores y devuelve un solo valor numerico

```
//incremento
let contador = 0;
contador++;
console.log(contador)           // 2

//decremento
let contador = 5;
contador--;
console.log(contador)           // 4
```

Aquí están los operadores adicionales:

- Residuo (%): Devuelve el residuo de una división.
- Incremento (++): Aumenta en uno el valor de una variable.
- Decremento (--): Disminuye en uno el valor de una variable.
- Negación unaria (-): Convierte un valor en su opuesto negativo.
- Positivo unario (+): Convierte un valor en su forma positiva
- Operador de exponenciación (\*\*): Calcula el exponente de un número.

## Operadores lógicos

Los Operadores Lógicos permiten crear condiciones compuestas al combinar dos o más condiciones. Estos operadores son fundamentales en la construcción de expresiones lógicas que determinan el flujo de un programa según diversas situaciones y estados de los datos.

Y (&&)

```
const valor1 = true;
const valor2 = false;
console.log(valor1 && valor2)           // false
```

- AND lógico (&&)

O(II)

```
const valor1 = true;
const valor2 = false;
console.log(valor1 || valor2) // true
```

- OR lógico (||)

NEGACION(!)

```
const isVisible = true;
console.log(!isVisible) // false
```

- NOT lógico (!)

## Ejemplos

<https://francescricart.com/metodos-javascript/>

```
<script>
var persona = {
  nombre: "Francesc",
  apellidos: "Ricart Muñoz",
  edad: 30,
  crecer: function(anos){this.
    edad += anos}
}

// persona.crecer(2) hará
// incrementará el valor de edad a 32
</script>
```

Ejemplo de método javascript personalizado

```
//Incremento
let contador = 0;
contador++;
console.log(contador) // 2

//decremento
let contador = 5;
contador--;
console.log(contador) // 4
```